

APPENDIX A
THEORETICAL PROOFS

A. Proof of Theorem 1

Proof. We consider standard gradient updates:

$$\begin{aligned}\mathbf{w}_{t+1}^\tau &= \mathbf{w}_t^\tau - \eta \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau), \\ \mathbf{w}_{t+1}^s &= \mathbf{w}_t^s - \eta \nabla \mathcal{L}^s(\mathbf{w}_t^s).\end{aligned}\quad (17)$$

Then, the difference between the two trajectories at step $t+1$ can be calculated as:

$$\begin{aligned}\mathbf{w}_{t+1}^s - \mathbf{w}_{t+1}^\tau &= (\mathbf{w}_t^s - \eta \nabla \mathcal{L}^s(\mathbf{w}_t^s)) - (\mathbf{w}_t^\tau - \eta \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau)) \\ &= (\mathbf{w}_t^s - \mathbf{w}_t^\tau) - \eta (\nabla \mathcal{L}^s(\mathbf{w}_t^s) - \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau)).\end{aligned}\quad (18)$$

Taking the norm, we have:

$$\|\mathbf{w}_{t+1}^s - \mathbf{w}_{t+1}^\tau\| \leq \|\mathbf{w}_t^s - \mathbf{w}_t^\tau\| + \eta \|\nabla \mathcal{L}^s(\mathbf{w}_t^s) - \nabla \mathcal{L}^\tau(\mathbf{w}_t^\tau)\|.\quad (19)$$

Finally, we apply Eq. (19) recursively from $t=0$ to $t=T-1$, noting that $\mathbf{W}_0^s = \mathbf{W}_0^\tau$, we obtain:

$$\|\mathbf{w}_T^s - \mathbf{w}_T^\tau\| \leq \eta \sum_{k=0}^{T-1} \|\nabla \mathcal{L}^s(\mathbf{w}_k^s) - \nabla \mathcal{L}^\tau(\mathbf{w}_k^\tau)\|.\quad (20)$$

□

B. Proof of Theorem 2

Proof. As the gradient matching objective is commonly calculated for each class separately, we define the class-wise loss for $\mathcal{T}$ and $\mathcal{S}$ as:

$$\begin{aligned}\mathcal{L}_{\text{GM}} &= \sum_{i=0}^{n-1} \left\| \frac{1}{|n_i|} \nabla \mathcal{L}_i^\tau(\mathbf{W}) - \frac{1}{|n'_i|} \nabla \mathcal{L}_i^s(\mathbf{W}) \right\|^2 \\ &= \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} (\Phi_i^\top \Phi_i \mathbf{W} - \Phi_i^\top Y_i) - \frac{1}{|n'_i|} (\Phi_i'^\top \Phi_i' \mathbf{W} - \Phi_i'^\top Y_i') \right\|^2 \\ &= \sum_{i=0}^{C-1} \left\| \left(\frac{1}{|n_i|} \Phi_i^\top \Phi_i - \frac{1}{|n'_i|} \Phi_i'^\top \Phi_i' \right) \mathbf{W} - \left(\frac{1}{|n_i|} \Phi_i^\top Y_i - \frac{1}{|n'_i|} \Phi_i'^\top Y_i' \right) \right\|^2 \\ &\leq \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} \Phi_i^\top Y_i - \frac{1}{|n'_i|} \Phi_i'^\top Y_i' \right\|^2 + \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} \Phi_i^\top \Phi_i - \frac{1}{|n'_i|} \Phi_i'^\top \Phi_i' \right\|^2 \|\mathbf{W}\|^2 \\ &= \|\mathbf{P}\Phi(X) - \mathbf{P}'\Phi(X')\|^2 + \sum_{i=0}^{C-1} \left\| \frac{1}{|n_i|} \Phi_i^\top \Phi_i - \frac{1}{|n'_i|} \Phi_i'^\top \Phi_i' \right\|^2 \|\mathbf{W}\|^2.\end{aligned}\quad (21)$$

This decomposition reveals that distribution matching, when combined with second-order embedding alignment, provides an upper bound for class-wise gradient matching. □

C. Proof of Theorem 3

Proof. Recall that MCKP is as follows: Given m mutually disjointed groups, where group G_i contains a set of K_i items $\{x_{i,1}, x_{i,2}, \dots, x_{i,K_i}\}$. Each item $x_{i,k} \in G_i$ has a value $v_{i,k} \in \mathbb{R}_{>0}$ and a weight $w_{i,k} \in \mathbb{Z}_{>0}$. The capacity of the knapsack is W . MCKP aims to select exactly one item from each group such that the total weight $\sum_i w_{i,\sigma(i)} \leq W$ is within the budget and the total value $\sum_i v_{i,\sigma(i)}$ is maximized. Here, the mapping $\sigma : \{1, 2, \dots, m\} \rightarrow \mathbb{Z}_{>0}$ specifies the index of the selected item in each group, i.e., $x_{i,\sigma(i)} \in G_i$ denotes the item chosen for group i . Based on this, we construct an instance of class-adaptive cluster allocation as: there are m classes of samples which corresponds to the number of groups in MCKP. The candidate set of cluster numbers for class i is denoted as $\{n'_{i,1}, n'_{i,2}, \dots, n'_{i,K_i}\}$, where $n'_{i,k} = k$

and $K_i = \max\{N' - (C-1), n_i\}$. This defines the discrete search space for each class i , which is structurally equivalent to the item set G_i in MCKP. Each element in this set corresponds to an item in MCKP, where the sample count $n'_{i,k}$ also plays a role of the item's weight $w_{i,k}$. Assume that we enumerate all candidate $n'_{i,k}$, and compute corresponding optimal clustering loss $\mathcal{L}(\{S_j^{i,k}\}^*)$, which corresponds to the negative value $-v_{i,k}$ in MCKP. The total cluster budget is set to W , satisfying that $\sum_i w_{i,\sigma(i)} = W$. From this instance, we can derive a perfect matching for the MCKP problem. This completes the NP-hardness proof. □

D. Proof of Theorem 4

Proof. In Algorithm 1, the initialization stage incurs its main cost in the ClassWiseClustering (Algorithm 2), invoked at line 6 of Algorithm 1, where each class i performs RunK-Means on $X_i \in \mathbb{R}^{n_i \times F}$ for I iterations. This step requires $\mathcal{O}(n_i n'_i I F)$ time, where $n'_i = \lfloor n_i \cdot r \rfloor$. Hence, the overall initialization complexity is $\mathcal{O}(IF \sum_{i=0}^{C-1} n_i \lfloor n_i r \rfloor)$. After initialization, the algorithm iteratively alternates between SoftAllocate (Algorithm 3, invoked at line 11 of Algorithm 1), which runs in constant time, and ClassWiseClustering (line 12), which dominates the iteration cost. In the worst case, clustering must be recomputed for all classes in each iteration. Let $\bar{n}' = \frac{1}{C} \sum_{i=0}^{C-1} n'_i = \frac{N'}{C}$ denote the average number of clusters per class. Then, over T iterations, the total cost of the search phase is $\mathcal{O}(T \sum_{i=0}^{C-1} n_i \bar{n}' I F) = \mathcal{O}(T \cdot \frac{NN'IF}{C})$. After convergence, Algorithm 1 performs RunKMeans(X_i, n'_i) for each class (line 22) to obtain the final partitions $\{\{S_j^i\}^*\}$. This post-processing cost is negligible when T is large. The time complexity of the search over T steps is $\mathcal{O}(T \cdot \frac{NN'IF}{C})$. Overall, the time complexity of HFILS is $\mathcal{O}(IF \sum_{i=0}^{C-1} n_i \cdot \lfloor n_i \cdot r \rfloor + T \cdot \frac{NN'IF}{C}) = \mathcal{O}(T \cdot \frac{NN'IF}{C})$. □

APPENDIX B
RELATED WORK

A. Coreset selection

Coreset selection methods aim to select a representative subset that achieves comparable model performance to training on the full dataset. Existing methods primarily rely on intermediate objectives that enforce distributional consistency between the subset and the original dataset [62], [63], [68]–[70]. Representative approaches include herding [62], which greedily adds samples to minimize the feature-space distance between the centers of the subset and the full dataset, and k -center [63], which iteratively selects samples to maximize spatial coverage. More recently, some methods have leveraged training signals such as gradients, loss values, or forgetting events to guide the selection of informative samples [20], [52], [71]–[73]. Despite their effectiveness, coreset selection methods share inherent limitations: since they can only select a small subset of existing samples, they are constrained by the original dataset and cannot generate new, potentially more representative instances [23]–[25]. Therefore, they fail to fully exploit the rich information in the full dataset, inevitably

leading to substantial information loss [71]. This problem is further amplified when informative signals are broadly distributed rather than concentrated in a few samples [52].

APPENDIX C COMPARISON WITH CGC

A. Difference in problem definition

The underlying problem definition and structural assumptions in our work differ fundamentally from those in [32]. Specifically, [32] focuses on graph data condensation, aiming to distill a large-scale graph into a compact yet informative one while preserving training utility for deep graph models. Accordingly, structural dependencies among entities are explicitly modeled by the adjacency matrix A and retained in the condensed graph [74], [75]. In contrast, our formulation targets tabular data composed of independent and identically distributed samples with heterogeneous feature types. We explicitly distinguish numerical and categorical attributes, denoted as $x_{i,j}^{num}$ and $x_{i,j}^{cat}$, respectively, and aim to condense feature-heterogeneous tabular samples for efficient training of deep tabular models without modeling any inter-sample structural information.

B. Difference in theoretical framework

Our theoretical analysis process for establishing a unified condensation framework differs from [32]. [32] unifies performance matching, parameter matching, and distribution matching in graph condensation, treating gradient matching as a representative instance of parameter matching and establishing DM as a proxy for PM. In contrast, we focus on independent-sample condensation and provide a finer-grained analysis of parameter matching by explicitly distinguishing gradient matching and training trajectory matching. We show that step-wise gradient discrepancies upper-bound long-range trajectory misalignment and further prove that DM upper-bounds GM, thereby yielding a unified framework encompassing MTT, GM, and DM.

C. Difference in label allocation strategies

Our class partitioning objective and optimization space differ from [32]. [32] focuses on graph condensation and performs partitioning on structure-aware node representations obtained via linear message passing. In contrast, we target tabular data condensation and, under a linearization simplification, conduct partitioning directly in the original sample space, since no structural dependencies exist among samples. Moreover, [32] considers only within-class partitioning with a fixed number of condensed samples per class, whereas we jointly optimize inter-class label allocation and intra-class clustering, leading to the Class-Adaptive Cluster Allocation Problem (CCAP), which adaptively determines the number of condensed samples per class.

APPENDIX D AUTOENCODER FOR STRING-TYPE FEATURE ENCODING

A. Computational Cost Analysis

To assess the computational efficiency of the autoencoder module, we empirically evaluate its runtime on three repre-

sentative tabular datasets with string-type categorical features, namely Adult (AD), Diabetes130US (DA), and Airlines (AI). The results show that the encoding process is highly efficient. In practice, the model converges within a small number of epochs and is therefore trained for a fixed 10 epochs as an offline preprocessing step. Under this setting, the runtime is 6.8 seconds on AD, 46.3 seconds on DA, and 98.5 seconds on AI. Overall, the proposed autoencoder serves as a lightweight and efficient preprocessing component that is trained only once per dataset, incurring a low computational cost that is negligible compared to the training overhead of prior learning-based condensation methods such as GM and DM.

APPENDIX E MORE EXPERIMENTAL SETTINGS

A. Detailed Datasets Descriptions

To evaluate the effectiveness and efficiency of our approach, we conduct experiments on 10 publicly available tabular datasets from the widely adopted OpenML benchmarks [61]. These datasets cover diverse real-world domains, including but not limited to energy forecasting, income prediction, and flight delay analysis, and encompass heterogeneous data types, such as numerical and categorical features. Their sizes range from the order of 10^4 to 10^6 instances, ensuring a comprehensive evaluation across small- to large-scale settings. In addition, we include datasets with different label distributions, reflecting varying degrees of class imbalance commonly observed in real-world data. To measure this imbalance, we adopt the widely used metric known as the ratio of minimum to maximum class frequency [64], where smaller values indicate a higher degree of imbalance. For all datasets, we randomly split them for 80%/10%/10% as train/validation/test sets. Table I provides the detailed statistics of all datasets.

B. Detailed Baselines Descriptions

We compare our method against a set of representative baselines, including coreset selection approaches (Random, Herding [62], and K-Center [63]) and dataset condensation methods (DM [25], GM [23], and MTT [24]).

- **Random.** Random selects a subset of training samples uniformly at random from the original data as the coreset.
- **Herding** [62]. Herding is a greedy algorithm that iteratively selects samples closest to the cluster center.
- **K-Center** [63]. K-Center picks multiple center points such that the largest distance between a data point and its nearest center point is minimized.
- **DM** [25]. DM learns condensed dataset by matching the feature embeddings between real and synthetic samples.
- **GM** [23]. GM proposes to infer the synthetic dataset by matching the gradients of a model trained on synthetic dataset to that on the original dataset.
- **MTT** [24]. MTT builds a condensation dataset to match the parameter trajectory of full dataset training.

C. Evaluation Metrics Details

To evaluate the classification performance of models trained on condensed datasets, we adopt two widely used metrics: *Accuracy* and *Macro-F1*. Accuracy is defined as the proportion of correctly predicted instances:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (22)$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively. To further provide a balanced view of classification performance across all classes, we utilize Macro-F1, which is the harmonic mean of precision and recall, computed independently for each class and then averaged:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=0}^{C-1} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \quad (23)$$

where $\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$ and $\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$.

D. Implementation Details

The pipeline of dataset condensation has two stages: (1) **Condensation step**, where we apply the DC methods on the training set to obtain small-scale synthetic data. (2) **Evaluation step**, where we train deep models on the condensed data from scratch and then evaluate their performance on the test set of the original data. For existing DC methods (i.e., DM, GM, and MTT) that requires model training, we use FT-Transformer [15], an effective and widely adopted tabular learning method, as the relay model. Our effectiveness experiments consider three reduction rates (1%, 0.1%, and 0.01%), with 0.1% used as the default setting in subsequent experiments unless stated otherwise. For C²TC, we set the maximum iteration to $T = 1000$, the tolerance to $\epsilon = 0.01$, and the early-stopping patience to $p = 10$. The adaptive weighting exponent γ is tuned over $\{0.25, 0.5, 0.75, 1.0\}$. Besides, we tune the step decay l in $\{0.1, 0.3, 0.5, 0.7, 0.9\}$. For all baselines, we report results using the official implementations with default hyperparameters specified in the original papers, and apply label encoding for categorical features. Methods that exceed the runtime limit (24 hours during condensation, or 5 days for expert trajectories generation during pre-processing in MTT) are denoted as OOT (out of time), while those that fail due to memory allocation errors are denoted as OOM (out of memory). The experiments are conducted on a Linux server with an Intel(R) Xeon(R) Gold 6330 CPU, a NVIDIA A800 GPU, and 128GB of RAM. To eliminate randomness, each experiment is repeated five times, and the average results with standard deviation are reported.

APPENDIX F

ADDITIONAL EXPERIMENTAL RESULTS

A. Additional Performance Analysis

We observe that on MI, C²TC already recovers over 93% accuracy of the full-data performance at $r = 0.01\%$, with

further increasing r to 1% yielding only marginal gains (e.g., from 53.2% to 54.4%). This behavior can be attributed to the following reasons. First, benefiting from class-adaptive clustering, C²TC is able to extract highly representative samples even at extremely low reduction rates. Second, the large-scale MI dataset exhibits substantial redundancy, allowing most informative patterns to be captured at very low reduction rates and leaving limited room for further improvement. Moreover, although increasing r from 0.01% to 1% corresponds to a 100 $\times$ relative increase, the absolute data scale remains small, resulting in only a modest performance gain. Based on these observations, we recommend small r values (e.g., $r = 0.01\%$) when computational or memory budgets are limited, and $r = 1\%$ as a robust default when a better balance between performance and generalization is desired.

To assess the reliability of the observed improvements in accuracy (Acc) and Macro-F1 (MF1), we conduct paired t-tests [76] comparing C²TC with the strongest coreset and condensation baselines, namely Herding and MTT, at $r = 0.1\%$, and report the corresponding p-values. Smaller p-values indicate stronger evidence that the observed performance differences are not due to random variation, and results with $p < 0.05$ are considered statistically significant. The tests were performed over 25 paired experimental runs per dataset, obtained from 5 independently generated synthetic datasets, each under 5 random seeds. As shown in Table VI, C²TC achieves statistically significant gains ($p < 0.05$) in accuracy on 9 out of the 10 datasets against Herding and 8 out of the 9 available datasets against MTT (excluding EP due to OOT), as well as in Macro-F1 on 8 out of the 10 and 8 out of the 9 datasets, respectively. In the few cases where improvements are not statistically significant (e.g., on the JA dataset against Herding as measured by accuracy), our method remains statistically comparable to the coreset baseline while still significantly outperforming the condensation competitor (MTT, $p = 3.75 \times 10^{-3}$).

TABLE VI: Paired t-test results (p -values) comparing C²TC against competitive baselines (Herding and MTT). Bold values indicate statistical significance ($p < 0.05$). Results for MTT on EP are omitted due to OOT.

Dataset	C ² TC vs. Herding		C ² TC vs. MTT	
	Acc p -val	MF1 p -val	Acc p -val	MF1 p -val
EL	6.90×10^{-33}	7.34×10^{-30}	3.78×10^{-15}	8.79×10^{-15}
AD	4.01×10^{-17}	3.20×10^{-20}	5.37×10^{-13}	5.13×10^{-10}
JA	4.76×10^{-1}	1.44×10^{-6}	3.75×10^{-3}	2.48×10^{-2}
DA	4.45×10^{-9}	1.05×10^{-13}	9.07×10^{-8}	1.78×10^{-12}
RS	2.28×10^{-2}	5.56×10^{-2}	2.18×10^{-2}	3.60×10^{-2}
EP	1.31×10^{-21}	1.86×10^{-19}	—	—
AI	1.47×10^{-32}	8.37×10^{-26}	1.99×10^{-22}	3.74×10^{-14}
CO	3.07×10^{-25}	4.79×10^{-20}	3.39×10^{-13}	8.42×10^{-15}
HI	5.18×10^{-17}	2.11×10^{-22}	6.82×10^{-10}	2.55×10^{-7}
MI	2.95×10^{-3}	8.84×10^{-2}	3.25×10^{-1}	5.62×10^{-2}

B. Additional Ablation Study

Categorical feature encoding. We evaluate the effectiveness of the proposed hybrid categorical feature encoding (HCFE) by comparing it with three widely adopted encoding strategies [55]: (i) label encoding, (ii) one-hot encoding followed by PCA, where PCA is applied to match feature dimensionality for fair comparison, and (iii) target encoding. All encoding strategies are evaluated on three datasets with varying scales (AD, DA, and AI). As reported in Table VII, HCFE consistently achieves the best performance across all datasets. For example, on the AD dataset, HCFE improves accuracy by 13.8% and Macro-F1 by 7.9% compared to Label Encoding, and also outperforms the most competitive baselines, including Target Encoding and One-hot + PCA, in both metrics. These results demonstrate that HCFE achieves superior accuracy and class-level balance, validating its effectiveness as an encoding strategy for condensed tabular data. Furthermore, the successful execution of the condensation process with these diverse baselines indicates that our framework is encoder-agnostic and can accommodate alternative numerical representations.

TABLE VII: Comparison of categorical feature encoding strategies for data condensation.

Dataset	Label		One-hot + PCA		Target		HCFE	
	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1
AD	67.7	62.9	74.4	70.0	78.8	69.9	81.5	70.8
DA	50.2	24.3	51.3	30.8	47.1	33.5	51.8	34.4
AI	53.1	52.0	61.0	56.0	59.1	58.2	61.5	58.4

C. Impact of Categorical Feature Imbalance

To evaluate the robustness of our categorical feature encoding method (HCFE) under severe feature imbalance commonly observed in real-world tabular datasets, we conduct experiments to examine the effects of varying imbalance levels on the proposed encoding and downstream model performance across three datasets with categorical features: AD, DA, and AI. Specifically, for each dataset, we select a categorical feature and construct training sets with controlled imbalance ratios of

1:1, 10:1, 100:1, and 1000:1 between two category values. For datasets containing a binary categorical feature, we directly use that column (e.g., sex in AD and diabetesMed in DA). For datasets without binary categorical features (e.g., AI), we select a multi-valued categorical column (DayOfWeek) and divide its seven unique values into two groups, assigning the first four days to 0 and the remaining three days to 1, yielding a binary feature on which controlled imbalance ratios can be imposed. These imbalance levels are achieved via a combination of label-conditioned downsampling and controlled upsampling on the selected feature. To ensure a fair and controlled comparison, all imbalance settings use the same number of training samples, strictly preserve the original label distribution, and share an identical test set, thereby isolating the effect of feature imbalance from other confounding factors. We evaluate downstream performance following the same condensation and learning pipeline using HCFE and the two baseline encoders. Figure 7 reports both accuracy and Macro-F1, where the x-axis denotes the feature imbalance level (e.g., 1 corresponds to 1:1, 10 to 10:1). HCFE demonstrates consistently stable performance under different degrees of categorical feature imbalance on all three datasets. For example, on the AI dataset, the accuracy remains within a narrow range of 60.21%–61.97%, while Macro-F1 varies only between 58.76% and 59.81%. This limited performance fluctuation suggests that HCFE preserves stable representation quality even under extremely skewed categorical distributions.

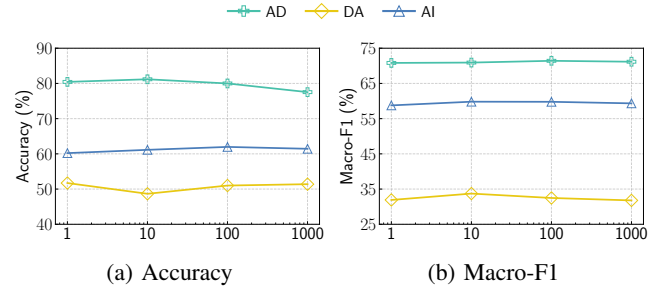


Fig. 7: Impact of categorical feature imbalance.